NUMERICAL METHODS

Unit-2: Solution of Equations

Comprehensive Study Notes

2.1 Introduction

This unit deals with finding the **roots** (solutions) of equations.

Types of Equations

Type	Description	Example
Algebraic	Only algebraic operations	$x^3 - 2x + 1 = 0$
Transcendental	Contains non-algebraic functions	$\sin x + e^x = 0$
Polynomial	Sum of terms with powers	$x^4 + 3x^2 - 2 = 0$

2.2 Bisection Method

The **Bisection Method** is a bracket method that repeatedly halves the interval containing the root.

Steps

1. Find **a and b** such that f(a) and f(b) have **opposite signs**
2. **Calculate midpoint**: c = (a + b)/2
3. **Evaluate f(c)**
4. **Select subinterval** where sign change occurs
5. **Repeat** until desired accuracy

Bisection Formula

> $$c = (a + b) / 2$$

Algorithm

```
1. Input: f(x), a, b, tolerance, max iterations
2. If f(a) x f(b) ≥ 0, no root found, exit
3. For i = 1 to max iterations:
   c = (a + b) / 2
   If f(c) = 0 or (b-a)/2 < tolerance, exit
   If f(a) x f(c) < 0, set b = c
   Else set a = c
4. Output: Root = c
```

Example: Find root of $f(x) = x^3 - x - 2 = 0$

```
Step 1: Find a and b with opposite signs
f(1) = 1 - 1 - 2 = -2
f(2) = 8 - 2 - 2 = 4
```

```
Since f(1) < 0 and f(2) > 0, root lies between 1 and 2

Iterations:
Iter | a      | b      | c        | f(c)                      | Interval
-----+--------+--------+----------+---------------------------+--------------
1    | 1      | 2      | 1.5      | -0.125                    | (1.5, 2)
2    | 1.5    | 2      | 1.75     | 1.609                     | (1.5, 1.75)
3    | 1.5    | 1.75   | 1.625    | 0.664                     | (1.5, 1.625)
4    | 1.5    | 1.625  | 1.5625   | 0.2515                    | (1.5, 1.5625)
5    | 1.5    | 1.5625 | 1.53125  | 0.05875                   | (1.5, 1.53125)
6    | 1.5    | 1.53125|1.515625| -0.0336                    | (1.5156, 1.5313)
7    | 1.5156 | 1.5313 | 1.5234   | 0.0116                    | (1.5156, 1.5234)
8    | 1.5156 | 1.5234 | 1.5195   | -0.0115                   | (1.5195, 1.5234)
9    | 1.5195 | 1.5234 | 1.5215   | 0.0005                    | Root ~ 1.5215
```

Error Formula

> $|Error| \leq (b - a) / 2^n$

Where a, b = initial interval, n = number of iterations.

Example: Iterations needed for error < 0.001

```
(2 - 1) / 2^n ≤ 0.001
1 / 2^n ≤ 0.001
2^n ≥ 1000
n ≥ 10 iterations
```

Advantages and Disadvantages

Advantages	Disadvantages
Always converges	Slow convergence
Simple to implement	Requires bracket with sign change
Guaranteed error bounds	Cannot find multiple roots

Robust	More iterations needed

2.3 Regula-Falsi Method (False Position Method)

The **Regula-Falsi Method** uses a line joining f(a) and f(b) to find the next approximation.

Formula

$$c = a - f(a) \times (b - a) / (f(b) - f(a))$$
$$= (a\, f(b) - b\, f(a)) / (f(b) - f(a))$$

Algorithm

```
1. Input: f(x), a, b, tolerance, max iterations
2. If f(a) x f(b) ≥ 0, no root found, exit
3. For i = 1 to max iterations:
   c = (a f(b) - b f(a)) / (f(b) - f(a))
   If |f(c)| < tolerance, exit
   If f(a) x f(c) < 0, set b = c
   Else set a = c
4. Output: Root = c
```

Example: Find root of $f(x) = x^3 - x - 2 = 0$

```
f(1) = -2, f(2) = 4 (opposite signs)

Iteration 1:
c = (1x4 - 2x(-2)) / (4 - (-2)) = (4 + 4) / 6 = 1.3333
f(1.3333) = 2.370 - 3.3333 = -0.9633

Iteration 2:
a = 1.3333, b = 2
c = (1.3333x4 - 2x(-0.9633)) / (4 + 0.9633)
f(1.4627) = 3.128 - 3.4627 = -0.3347
```

```
Iteration 3:
a = 1.4627, b = 2
c = (1.4627x4 - 2x(-0.3347)) / (4 + 0.3347)
f(1.5040) = 3.401 - 3.504 = -0.103

Iteration 4:
a = 1.5040, b = 2
c = (1.5040x4 - 2x(-0.103)) / (4 + 0.103)
f(1.5163) = 3.486 - 3.5163 = -0.0303

Root ~ 1.5215 (after more iterations)
```

Bisection vs Regula-Falsi

Feature	Bisection	Regula-Falsi
Convergence	Always	Usually
Speed	Slow	Faster
Formula	Midpoint	Chord intersection
Error Bound	Guaranteed	Not guaranteed
Use	When reliability needed	When speed needed

2.4 Newton-Raphson Method

The **Newton-Raphson Method** is an open method that uses derivatives to find roots.

Formula

> $$x_{n+1} = x_n - f(x_n) / f'(x_n)$$

Algorithm

```
1. Input: f(x), f'(x), x₀, tolerance, max iterations
2. For i = 1 to max iterations:
   x₁ = x₀ - f(x₀) / f'(x₀)
   If |x₁ - x₀| < tolerance, exit
   x₀ = x₁
3. Output: Root = x₁
```

Example: Find root of f(x) = x^3 - x - 2 = 0

```
f(x) = x³ - x - 2
f'(x) = 3x² - 1
x₀ = 1.5

Iteration 1:
x₀ = 1.5
f(1.5) = 3.375 - 3.5 = -0.125
f'(1.5) = 6.75 - 1 = 5.75
x₁ = 1.5 - (-0.125/5.75) = 1.5 + 0.021739 = 1.521739

Iteration 2:
x₁ = 1.521739
f(1.521739) = 1.521739³ - 1.521739 - 2
f'(1.521739) = 3(1.521739)² - 1
```

```
x₂ = 1.521739 - f(1.521739)/f'(1.521739)

Iteration 3:
x₂ ~ 1.521527
f(1.521527) ~ 0.0000001
f'(1.521527) ~ 5.945
x₃ ~ 1.521527

Root ~ 1.521527
```

Convergence Rate

Newton-Raphson has **quadratic convergence**:

> $$|x_{n+1} - x_n| \leq M|x_n - x_{n-1}|^2$$

Advantages and Disadvantages

Advantages	Disadvantages
Very fast convergence	Requires derivative
Quadratic convergence	May diverge
Elegant formula	May fail if f'(x) = 0
Works for complex roots	Sensitive to initial guess

Comparison of Root-Finding Methods

Method	Type	Speed	Reliability	Derivative Needed
Bisection	Bracket	Slow	Very High	No
Regula-Falsi	Bracket	Medium	High	No
Newton-Raphson	Open	Fast	Low	Yes

2.5 Solution of Simultaneous Linear Equations

A system of n linear equations in n unknowns:

> $a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1$
>
> $a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n = b_2$
>
> ...
>
> $a_{n1}x_1 + a_{n2}x_2 + \ldots + a_{nn}x_n = b_n$

2.6 Gauss Elimination Method

Transforms the system into an **upper triangular form** using row operations, followed by **back substitution**.

Example: Solve

> $2x + 3y + z = 9$
>
> $x + 2y + 3z = 10$
>
> $3x + y + 2z = 11$

Step 1: Augmented Matrix

```
[ 2   3   1  |  9 ]
[ 1   2   3  | 10 ]
[ 3   1   2  | 11 ]
```

Step 2: Forward Elimination

```
Eliminate x from row 2 and 3:
Row 2 = Row 2 - (1/2)Row 1
Row 3 = Row 3 - (3/2)Row 1

[ 2    3     1   |  9  ]
[ 0    0.5   2.5 |  5.5]
[ 0   -3.5   0.5 | -2.5]

Eliminate y from row 3:
Row 3 = Row 3 + 7 x Row 2

[ 2   3    1   |  9 ]
[ 0   0.5  2.5 | 5.5]
[ 0   0    18   | 36 ]
```

Step 3: Back Substitution

```
From row 3: 18z = 36  =>  z = 2
From row 2: 0.5y + 2.5(2) = 5.5 => 0.5y + 5 = 5.5 => y = 1
From row 1: 2x + 3(1) + 1(2) = 9 => 2x + 5 = 9 => x = 2

Answer: x = 2, y = 1, z = 2
```

2.7 Gauss-Jordan Method

Extends Gauss Elimination to transform to the **identity matrix**, giving the solution directly.

Example: Same system

```
Step 1: Augmented matrix
[ 2   3   1  |  9 ]
[ 1   2   3  | 10 ]
[ 3   1   2  | 11 ]

Step 2: Make diagonal elements = 1
Divide row 1 by 2:
[ 1   1.5  0.5 | 4.5 ]
[ 1   2    3   | 10  ]
[ 3   1    2   | 11  ]

Eliminate x from rows 2 and 3:
Row 2 = Row 2 - Row 1
Row 3 = Row 3 - 3xRow 1
[ 1   1.5   0.5 | 4.5 ]
[ 0   0.5   2.5 | 5.5 ]
[ 0  -3.5   0.5 | -2.5]

Divide row 2 by 0.5:
[ 1   1.5  0.5 | 4.5 ]
[ 0   1    5   | 11  ]
[ 0  -3.5  0.5 | -2.5]

Eliminate y from rows 1 and 3:
Row 1 = Row 1 - 1.5xRow 2
Row 3 = Row 3 + 3.5xRow 2
[ 1   0   -7  | -12 ]
[ 0   1    5  | 11  ]
[ 0   0    18 | 36  ]

Divide row 3 by 18:
[ 1   0   -7 | -12 ]
[ 0   1    5 | 11  ]
[ 0   0    1 | 2   ]

Eliminate z from rows 1 and 2:
```

```
Row 1 = Row 1 + 7xRow 3
Row 2 = Row 2 - 5xRow 3
[ 1   0   0 | 2 ]
[ 0   1   0 | 1 ]
[ 0   0   1 | 2 ]

Answer: x = 2, y = 1, z = 2
```

Gauss Elimination vs Gauss-Jordan

Feature	Gauss Elimination	Gauss-Jordan
Purpose	Upper triangular	Identity matrix
Steps	Forward + Backward	Forward + Backward (all rows)
Work	Less	More
Solution	Requires back substitution	Direct reading
Use	Solving systems	Matrix inversion

2.8 Jacobi Iterative Method

An iterative technique where each variable is solved using values from the **previous iteration**.

General Formula

> $$x_i^{(k+1)} = (1/a_{ii}) \left[b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right]$$

Example: Solve

> $$4x + y + z = 9$$
>
> $$x + 5y + z = 11$$
>
> $$x + y + 3z = 7$$

Solve each equation for x, y, z:
$x = (9 - y - z) / 4$
$y = (11 - x - z) / 5$
$z = (7 - x - y) / 3$

Initial guess: $x_0 = 0$, $y_0 = 0$, $z_0 = 0$

Iteration 1:
$x_1 = (9 - 0 - 0) / 4 = 2.25$
$y_1 = (11 - 0 - 0) / 5 = 2.20$
$z_1 = (7 - 0 - 0) / 3 = 2.33$

Iteration 2:
$x_2 = (9 - 2.20 - 2.33) / 4 = 1.117$
$y_2 = (11 - 2.25 - 2.33) / 5 = 1.284$
$z_2 = (7 - 2.25 - 2.20) / 3 = 0.850$

Iteration 3:

$x_3 = (9 - 1.284 - 0.850) / 4 = 1.717$

$y_3 = (11 - 1.117 - 0.850) / 5 = 1.807$

$z_3 = (7 - 1.117 - 1.284) / 3 = 1.533$

Continues... Exact solution is (1, 2, 1).

2.9 Gauss-Seidel Iterative Method

Similar to Jacobi but uses **updated** values immediately within the same iteration.

General Formula

> $$x_i^{(k+1)} = (1/a_{ii}) \left[b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right]$$

Example: Same system

```
x = (9 - y - z) / 4
y = (11 - x - z) / 5
z = (7 - x - y) / 3

Initial: x₀ = 0, y₀ = 0, z₀ = 0

Iteration 1:
x₁ = (9 - 0 - 0) / 4 = 2.25
y₁ = (11 - 2.25 - 0) / 5 = 1.75  (uses updated x)
z₁ = (7 - 2.25 - 1.75) / 3 = 1.00  (uses updated x, y)

Iteration 2:
x₂ = (9 - 1.75 - 1.00) / 4 = 1.5625
y₂ = (11 - 1.5625 - 1.00) / 5 = 1.6875
z₂ = (7 - 1.5625 - 1.6875) / 3 = 1.25

Iteration 3:
x₃ = (9 - 1.6875 - 1.25) / 4 = 1.5156
y₃ = (11 - 1.5156 - 1.25) / 5 = 1.6469
z₃ = (7 - 1.5156 - 1.6469) / 3 = 1.2792

Converging to: x ~ 1, y ~ 2, z ~ 1
```

Jacobi vs Gauss-Seidel

Feature	Jacobi	Gauss-Seidel
Values Used	Old values	Updated values
Convergence	Slower	Faster
Memory	More (store all)	Less
Parallelization	Easier	Harder
Suitability	Large systems	Most systems

Convergence Condition

System must be **diagonally dominant**:

> $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$

2.10 Complete Summary

Root Finding Methods Summary

Method	Formula	Type	Speed	Derivative?
Bisection	$c = (a + b)/2$	Bracket	Slow	No
Regula-Falsi	$c = (a\,f(b) - b\,f(a))/(f(b)-f(a))$	Bracket	Medium	No
Newton-Raphson	$x_{n+1} = x_n - f(x_n)/f'(x_n)$	Open	Fast	Yes

Linear Equation Methods Summary

Method	Type	Speed	Complexity
Gauss Elimination	Direct	Fast	Medium
Gauss-Jordan	Direct	Medium	High
Jacobi	Iterative	Slow	Low
Gauss-Seidel	Iterative	Medium	Low

2.11 Previous Year Questions

Short Questions (2 marks)

1. What is the difference between algebraic and transcendental equations?
2. State the bisection method formula.
3. Write the Newton-Raphson formula.
4. What is the condition for convergence in iterative methods?
5. What is the difference between Gauss Elimination and Gauss-Jordan methods?
6. What is diagonal dominance?
7. Why is Newton-Raphson method fast?
8. When does bisection method fail?

Medium Questions (5 marks)

1. Find the root of $x^3 - 2x - 5 = 0$ using bisection method (3 iterations).
2. Find the root of $x^3 - x - 1 = 0$ using Regula-Falsi method.
3. Find the root of $x^4 - x - 10 = 0$ using Newton-Raphson method.
4. Solve the system using Gauss Elimination: $2x + y + z = 5$, $x + 3y + z = 7$, $x + y + 2z = 6$.
5. Solve using Gauss-Seidel method: $3x + y + z = 5$, $x + 4y + z = 6$, $x + y + 5z = 7$.

Long Questions (12 marks)

1. (a) Explain the bisection method with algorithm and example.
   (b) Find the root of $x^3 - 4x - 9 = 0$ using Regula-Falsi method.

2. (a) Explain Newton-Raphson method with algorithm and example.
   (b) Compare bisection, Regula-Falsi, and Newton-Raphson methods.
3. (a) Solve the system using Gauss Elimination method: $10x + y + z = 12$, $2x + 10y + z = 13$, $2x + 2y + 10z = 14$.
   (b) Explain Jacobi and Gauss-Seidel methods with example.

Quick Reference

Method	Formula
Bisection	$c = (a + b)/2$
Regula-Falsi	$c = (a\,f(b) - b\,f(a))/(f(b) - f(a))$
Newton-Raphson	$x_{n+1} = x_n - f(x_n)/f'(x_n)$
Gauss Elimination	Forward elimination + Back substitution
Gauss-Jordan	Transform to identity matrix
Jacobi	$x_i^{(k+1)} = (b_i - \sum_{j \neq i} a_{ij} x_j^{(k)})/a_{ii}$
Gauss-Seidel	Uses updated values immediately